

## Introduction

---

Every real application talks to something outside itself — a weather service, a payment processor, a login provider, your own backend running on a different machine. The moment your code crosses that boundary, it stops being purely a Python problem and becomes a Python-plus-network problem. And networks fail constantly: servers time out, endpoints go down, connections drop mid-request. None of that shows up when you're testing locally against a service that's always up and always fast.

This matters for interviews too. When a company asks you to "call this API and process the results," they're rarely testing whether you know the syntax for `requests.get()`. They're watching whether you handle the response as if it might not be what you expect. A candidate who parses JSON directly off a raw response, with no status check and no timeout, is showing they've only ever tested against the happy path. That's the gap this lesson closes.

## Problem Overview

---

Say you write three lines of code: `import requests`, call `requests.get(url)`, and print the result. Run it against a server that's slow, unreachable, or just never responds, and your script doesn't crash — it just hangs. No traceback, no error message, nothing. The terminal sits there with a blinking cursor, possibly forever.

The underlying issue is that `requests.get()` makes no promises about how long it will wait for a response, and it makes no judgment about whether the response it got back is actually useful. A 404 page and a valid JSON payload both come back as ordinary response objects. It's entirely on you to check what actually happened.

## Example

---

```
import requests

response = requests.get("https://api.github.com/users/octocat")
print(response.status_code)    # 200
data = response.json()
print(data["url"])             # https://api.github.com/users/octocat
```

That works fine — until the URL is wrong, the endpoint moves, or the server is overloaded:

```
response = requests.get("https://api.github.com/users/does-not-exist-1234")
print(response.status_code)    # 404
data = response.json()         # still "succeeds" – parses an error body as JSON
```

Nothing here raises an exception. `response.json()` happily parses whatever text came back, whether it's a real user object or a GitHub error message saying "Not Found." If your code assumes `data["url"]` exists, you get a `KeyError` deep inside your program, far from where the actual problem occurred — a broken request that was never checked.

## Intuition

---

Think about what actually happens when you call `requests.get()`. Your script sends a request out over the network to a machine you don't control, running software you didn't write, on a connection with no guarantees. Three broad things can go wrong on the way back:

1. The server responds, but with bad news (404, 500, etc.) — the connection worked fine, the *content* is the problem.
2. The server never responds at all, but is at least reachable — this is a timeout.
3. There's no server to reach in the first place — your Wi-Fi drops, DNS fails, the connection is refused — this is a connection error.

An experienced engineer treats all three as expected outcomes, not exceptional ones. The fix isn't one trick — it's three independent guards, one for each failure mode, stacked in the same block of code.

## Brute Force Solution

---

The "brute force" version is really just the naive version most people write first: call `requests.get()`, assume it comes back, assume the status is 200, assume the body is valid JSON.

**Idea:** Fire the request, read the response, move on.

**Algorithm:** 1. Call `requests.get(url)`. 2. Call `.json()` on the result. 3. Use the data.

**Advantages:** Minimal code, fast to write, works fine in a demo against a healthy server.

**Disadvantages:** No timeout means an unresponsive server can hang the script indefinitely. No status check means error pages get treated as real data. No exception handling means any network hiccup crashes the program with a raw traceback — or worse, silently produces garbage.

**Time complexity:**  $O(1)$  beyond the network round trip itself — this isn't an algorithmic problem, it's a robustness problem. **Space complexity:**  $O(n)$  where  $n$  is the size of the response body.

## Optimal Solution

The optimal approach adds three specific safeguards, each addressing one of the failure modes from the intuition section:

| Failure mode                                  | Symptom without a guard                      | Guard                                                   |
|-----------------------------------------------|----------------------------------------------|---------------------------------------------------------|
| Slow/unresponsive server                      | Script hangs forever                         | <code>timeout=5</code>                                  |
| Bad status code (404, 500)                    | Error page treated as real data              | <code>response.raise_for_status()</code>                |
| No server reachable (dead Wi-Fi, DNS failure) | <code>ConnectionError</code> crashes program | <code>except requests.exceptions.ConnectionError</code> |
| Server too slow to answer within the limit    | <code>Timeout</code> crashes program         | <code>except requests.exceptions.Timeout</code>         |

The key insight worth calling out: a dropped connection and a slow server are *different* failures, even though both eventually raise an exception. A `Timeout` means the server exists and is just too slow. A `ConnectionError` means there was no server to talk to at all — your machine couldn't even establish the connection. Catching them separately means your error message actually matches the real cause, instead of one generic "something went wrong."

## Python Code

```
import requests

def fetch_json(url: str, timeout: float = 5.0) -> dict | None:
    """Fetch JSON from a URL, handling timeouts, bad status codes,
    and dropped connections. Returns None if the request fails."""
    try:
        response = requests.get(url, timeout=timeout)
        response.raise_for_status()
        return response.json()
    except requests.exceptions.Timeout:
        print(f"Request to {url} timed out after {timeout}s")
    except requests.exceptions.ConnectionError:
        print(f"Could not connect to {url} – check your network")
    except requests.exceptions.RequestException as exc:
        print(f"Request to {url} failed: {exc}")
```

```
    return None

if __name__ == "__main__":
    result = fetch_json("https://api.github.com/users/octocat")
    if result:
        print(result["url"])
```

## Code Walkthrough

---

- `timeout=timeout` on the `requests.get()` call caps how long the function waits for a response. Without it, there is no built-in limit — the request can hang forever.
- `response.raise_for_status()` inspects the status code and raises `requests.exceptions.HTTPError` if it's a 4xx or 5xx. A 200 passes through silently; anything else becomes a real exception instead of an error page disguised as data.
- The `except` blocks are ordered from most specific to least specific. `Timeout` and `ConnectionError` are caught separately so the printed message matches the actual cause. The final `except requests.exceptions.RequestException` is a catch-all for anything else in the requests exception hierarchy, including the `HTTPError` raised by `raise_for_status()`.
- The function returns `None` on failure rather than letting an exception propagate, so callers can check the result with a simple `if result:` instead of wrapping every call site in its own try block.

## Dry Run

---

Call `fetch_json("https://api.github.com/users/octocat")` against a healthy server:

1. `requests.get()` sends the request, server responds within 5 seconds with status 200.
2. `raise_for_status()` checks 200 — no exception raised.
3. `response.json()` parses the body into a dict.
4. The dict is returned; `result["url"]` prints successfully.

Now call it against a server that never responds:

1. `requests.get()` sends the request and starts waiting.
2. At 5 seconds, `requests` gives up and raises `requests.exceptions.Timeout`.
3. The `except Timeout` block catches it, prints a clear message.
4. The function returns `None`; the caller's `if result:` check skips the rest safely.

## Complexity Analysis

---

**Time:**  $O(1)$  in terms of Python-side computation — the dominant cost is the network round trip, which is bounded by the `timeout` value rather than being unbounded. This bound is exactly what turns an unpredictable hang into a predictable worst case.

**Space:**  $O(n)$ , where  $n$  is the size of the response body being held in memory and parsed into a dict. This is unavoidable — you can't process a response without holding it.

Both are correct because the guards added don't change what the function computes, only how it handles failure — the complexity of the happy path is untouched.

## Alternative Solutions

---

You could reach for `urllib`, which is in the standard library and needs no extra dependency. But you'd end up writing your own JSON header handling, your own exception mapping, and your own status-code checks — `requests` gives you all of that for free with cleaner syntax. For high-throughput async workloads, `httpx` or `aiohttp` are reasonable alternatives since they support `async/await`, but for straightforward synchronous API calls, `requests` remains the standard, well-understood default.

## Edge Cases

---

- **Empty response body:** `response.json()` raises `json.JSONDecodeError` if the body isn't valid JSON (e.g., an empty string) — worth catching separately if you expect this.
- **Very large responses:** memory usage scales with body size; consider `stream=True` for large downloads instead of loading everything at once.
- **Redirects:** `requests` follows redirects by default, which can mask an endpoint that's moved — check `response.history` if that matters to your use case.
- **Malformed JSON with a 200 status:** `raise_for_status()` won't catch this since the status code is fine; you still need a `try/except` around `.json()` itself in strict pipelines.
- **Extremely slow but eventually successful requests:** a short `timeout` can cause false failures on legitimately slow endpoints — tune the value to the API you're calling, not an arbitrary default.

## Common Mistakes

---

1. Omitting `timeout` entirely, leaving the request able to hang indefinitely.

2. Assuming a 200 status guarantees valid data instead of calling `raise_for_status()`.
3. Catching only `requests.exceptions.RequestException` and losing the distinction between a timeout and a dead connection.
4. Calling `.json()` before checking the status code, so an error page gets parsed as if it were real data.
5. Wrapping the entire script in one broad `except Exception`, which hides bugs unrelated to the network call.
6. Reusing a single hardcoded timeout value across calls to APIs with very different expected latencies.

## Interview Questions

---

- Why doesn't `requests.get()` raise an exception on a 404 or 500 by default?
- What's the difference between `requests.exceptions.Timeout` and `requests.exceptions.ConnectionError`?
- How would you add retry logic on top of this pattern for transient failures?
- What happens if you call `.json()` on a response that isn't valid JSON?
- How would you adapt this function to handle rate limiting (HTTP 429)?

## Similar Problems

---

- **Design a rate limiter** — tests similar judgment about handling external, unreliable systems gracefully.
- **Implement a retry-with-backoff decorator** — a natural extension of this pattern for transient failures.
- **Build a simple web scraper with error recovery** — applies the same timeout/status/exception structure at scale across many URLs.

## Key Takeaways

---

The network fails more often than your local machine ever does, and that's just a fact to design around, not an edge case to dismiss. Three additions turn a fragile script into a resilient one: a `timeout` so a dead server can't freeze your program, `raise_for_status()` so a bad response never gets mistaken for real data, and a `try/except` block so you decide what happens next instead of a stack trace deciding for you.

## Watch the Video

---

For the full walkthrough with live debugging and the exact moment this pattern gets built up step by step, watch the video here: {LINK}

## About the Series

---

This lesson is part of **Fun with Learning Technology**, a series built around one idea: understanding *why* code works beats memorizing *that* it works. Every episode takes a problem developers actually hit — in Python, in interviews, in production — and builds the intuition behind the fix, not just the fix itself.

## Call To Action

---

If this saved you from a bug you didn't know you had, drop a like on the video and subscribe on YouTube for more of this series. For deeper written breakdowns like this one, subscribe to the Substack. And if you've got a war story about a hang or a silent failure that taught you a lesson the hard way, share it in the comments — those are always the best ones.

---

## Quick Review

---

### What's the tell you need requests + JSON parsing over raw sockets?

Any 'fetch data from a URL' task — requests handles TCP/TLS/headers, `.json()` parses the body.

### Time complexity of a single `requests.get()` call?

$O(1)$  in your code; dominated by network latency, not algorithmic work.

### Why check status codes instead of trusting a 200-shaped response?

Some APIs return 200 with an error payload inside JSON — status lies by omission; always check content too.

### Bug: `response.json()` called without checking status first — what happens?

Raises `exceptions.JSONDecodeError` on error pages (HTML/empty body), masking the real HTTP failure.

**raise\_for\_status() vs manual status code checks?**

raise\_for\_status() raises HTTPError on 4xx/5xx automatically; manual checks need explicit if/elif for each code.

**Follow-up: why always pass a timeout to requests calls?**

Without one, a hung server blocks your program forever; timeout raises Timeout exception instead.

---

Python Lesson 31: Working with APIs and HTTP (Making requests, parsing JSON responses, and handling errors and timeouts)  
— free Python cheat sheet